

搜索高效的计算机化自适应测试 (SECAT) 精读笔记

原文: 3583780.3615049.pdf (CIKM '23, Hong et al.) · 代码: <https://github.com/bigdata-ustc/SECAT> (已 clone 到本目录 SECAT/) · 笔记于 2026-06-27

一、解决什么问题

1. 背景: CAT 的“选题”是核心, 但它慢

计算机化自适应测试 (CAT) 的工作循环很简单: 每一步根据学生当前能力估计 $\hat{\theta}_t$, 从题库里挑一道“最值得问”的题, 学生作答后再更新能力估计, 循环往复。选题靠的是一个打分函数 $S(\hat{\theta}, q)$ ——它衡量“这道题对当前这个学生有多大信息量”。最经典的是 **Fisher 信息量 (FSI)**: 越能把学生区分开、越贴近学生当前水平的题, 分数越高。

问题在于怎么找出分数最高的那道题。现有做法是暴力穷举: 把题库里 N 道题逐一算一遍 S , 取最大值。时间复杂度 $O(N)$ 。

2. 痛点: 题库一大, 穷举就崩

论文给的具体数字很扎眼: 一个 10 万规模的题库 (在线教育里很常见), 用 FSI 暴力选一道题, 在 2.20 GHz 的 CPU 上每一步要 10 秒以上。一次测试要选十几道题, 用户根本等不起。所以现实里 GMAT、GRE 这些机构只能让专家人工写规则先把候选缩小——费人力, 还限制了 CAT 的效果。

3. 思路: 借鉴推荐/检索里的“空间划分”, 但 CAT 有三个独特难点

推荐和信息检索里早就有成熟的加速套路: 把候选项分到若干子空间, 搜索时只看少数几个子空间, 复杂度从 $O(N)$ 降到 $O(\log N)$ (典型如 KD-Tree、Ball Tree、HNSW)。但直接搬到 CAT 行不通, 三个坎:

1. 划分必须“学生感知”。普通的树只按“题目之间像不像”来分; 但 CAT 真正关心的是“这道题适不适合某个学生”。应该把“适合同一类学生的题”分到一起, 而不是“长得像的题”分到一起。
2. 打分函数在欧氏空间里无效。Ball Tree 这类结构靠点积 / 欧氏距离的几何性质来剪枝。可 FSI (式 3, 一个带平方和除法的分式) 根本不是点积, 没有这种几何性质, 没法直接拿来建树、剪枝。
3. 多轮选题是相互依赖的。CAT 一道接一道地选, 而且 $\hat{\theta}$ 会越来越稳定 (渐近收敛到真值)。相邻几步 $\hat{\theta}$ 很接近, 意味着每步都从头搜整棵树是重复劳动。

SECAT 就是针对这三个坎, 分别给出: (1) 学生感知的空间划分; (2) 打分函数蒸馏 (双塔网络); (3) 利用渐近理论的多轮搜索。最终把每步选题从“算 10 万道题”降到“平均算 83 道题、<0.1 秒”, **200 倍以上加速, 精度几乎不掉**。

二、问题定义 (输入 / 输出)

系统层面: - 题库 $Q = \{q_1, \dots, q_N\}$, N 道题, 每题有校准好的参数 (IRT 下是难度 b_j 、区分度 a_j)。- CDM M : 根据学生历史作答估计能力 $\hat{\theta}$ 。IRT 里 θ 是一个一维实数。- 选题: $q_t^* = \arg \max_{q_j \in Q} S(\hat{\theta}_t, q_j)$ (式 2)。

SECAT 要做的事: 把这个 $\arg \max$ 从“扫遍 N 道题”变成“只看 $O(\log N)$ 道题”, 且选出的题尽量和暴力选出的一致。

一个具体小例子 (IRT 场景): 假设题库 4 道题, 难度分别 $b = [-2, -0.5, 0.5, 2]$, 区分度都 $a = 1$ 。某学生当前估计 $\hat{\theta} = 0.4$ 。- FSI 在 IRT 下大致在“题目难度 $\approx$ 学生能力”时最大 ($P(\hat{\theta}) \approx 0.5$ 时信息量峰值)。- 暴力做法: 对 4 道题各算一遍式 (3), 发现 $b = 0.5$ 那题的 $P(\hat{\theta})$ 最接近 0.5, 信息量最大 $\rightarrow$ 选它。- SECAT 做法: 建树时已把“适合中等能力学生的题” ($b = -0.5, 0.5$) 聚到了一个子空间; 测试时只进这个子空间、算 2 道题就够, 不碰 $b = -2$ 和 $b = 2$ 那两道明显不适的。

输出都是同一道题 $b = 0.5$, 但 SECAT 少算了一半 (题库大时是少算 99.9%)。

三、模型结构

SECAT 分两大阶段, 测试前 (离线) 建索引 + 测试中 (在线) 搜题。

离线 (建索引):

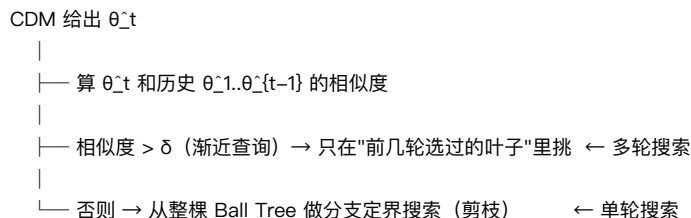
题库 Q ——[打分函数蒸馏: 双塔网络 f, g]——> 每道题变成向量 $g(q)$

学生能力变成向量 $f(\theta)$

题库 Q + 学生能力集 Ω ——[学生感知空间划分]——> Ball Tree (度量树)

每个叶子 = 一小撮“适合相似学生”的题

在线（每一步选题）：



三个模块逐一说：

模块 A：学生感知的空间划分（建树）——解决难点 (1)

普通 Ball Tree 建树 (Andrew Moore 的经典做法)：每个节点选距离最远的两个点 A, B 当质心，用垂直平分超平面把点分两半，递归下去。问题是“距离最远”只看题目本身，不看学生。

SECAT 的改法：选质心时，不选“最远的两道题”，而选“最适合能力最高的学生的题” A 和“最适合能力最低的学生的题” B 。这样超平面分出来的两半，天然就是“适合高能力学生的题”vs“适合低能力学生的题”。递归下去，每个叶子就是“适合某个能力区间学生”的一小撮题。(细节见公式详解的算法 1)

模块 B：打分函数蒸馏（双塔网络）——解决难点 (2)

FSI/KLI/EMC 这些打分函数不是点积，没法建树。SECAT 用一个双塔网络去拟合原打分函数：- 用户塔 f ：把学生信息（能力、历史作答）编码成向量 $f(\theta)$ 。- 题目塔 g ：把题目参数（难度、区分度）编码成向量 $g(q)$ 。- 让 $f(\theta)^\top g(q) \approx S(\theta, q)$ 。

训练好后，“信息量”就近似变成了“点积”，于是 Ball Tree 的几何剪枝（点积上界）就能用了。注意：建树和剪枝用点积，但真正比较哪道题最好时，叶子内部仍用原版 S (FSI 等) 精确计算——这是 SECAT 比纯 ANN 精度高的关键（见 4.5 节）。

模块 C：搜索策略（单轮剪枝 + 多轮复用）——解决难点 (3)

单轮：深度优先 + 分支定界。每个球 B （子空间）内点积有个上界 $f(\theta)^\top q_c + r\|f(\theta)\|$ (q_c 球心、 r 半径)。如果某子空间的上界 $<$ 当前已找到的最大信息量，整个子空间直接剪掉不算。先进上界大的子空间，尽快找到好题、好剪枝。最终只碰 L 个叶子。

多轮：利用 $\hat{\theta}$ 随步数收敛的事实。若当前 $\hat{\theta}_t$ 和历史某个 $\hat{\theta}_{t'}$ 足够像（余弦相似度 $> \delta$ ，称“渐近查询”），就不再搜整棵树，直接从“那几轮选过的叶子”里挑。 $\hat{\theta}$ 越往后越稳，这种复用越频繁， L 进一步降到近似常数。

四、关键公式详解

公式 1：IRT 作答概率（式 1）

$$P(r_{i,j} | \theta_i, a_j, b_j) = \frac{1}{1 + e^{-1.7a_j(\theta_i - b_j)}}$$

- 符号： θ_i 学生 i 能力（一维实数）； b_j 题 j 难度； a_j 题 j 区分度；1.7 是让 logistic 贴近正态 ogive 的常数。
- 举例：学生 $\theta = 0.4$ ，题 $b = 0.5, a = 1$ 。指数项 $-1.7 \times 1 \times (0.4 - 0.5) = 0.17$ ， $P = 1/(1 + e^{0.17}) = 1/(1 + 1.185) = 0.458$ 。即这道难度跟他差不多的题，他约有 46% 概率答对——这种“五五开”的题信息量最大。换成 $b = -2$ 的简单题： $-1.7(0.4 + 2) = -4.08$ ， $P = 1/(1 + e^{-4.08}) = 0.983$ ，几乎必对，问了等于白问。
- 作用：CDM 的核心，既用来估 $\hat{\theta}$ ，也是 FSI 等打分函数的输入。

公式 2 & 3：选目标与 Fisher 打分函数（式 2、3）

$$q_t^* = \arg \max_{q_j \in Q} S(\hat{\theta}_t, q_j), \quad S(\hat{\theta}) = \frac{[P'(\hat{\theta})]^2}{P(\hat{\theta})(1 - P(\hat{\theta}))}$$

- 符号： $P = P(\hat{\theta})$ 答对概率（式 1）， P' 是它对 θ 的导数。IRT 下 $P' = 1.7a P(1 - P)$ ，代入得 $S = (1.7a)^2 P(1 - P)$ 。
- 举例：上面 $b = 0.5$ 那题 $P = 0.458$ ， $S = (1.7)^2 \times 0.458 \times 0.542 = 2.89 \times 0.248 = 0.717$ 。 $b = -2$ 那题 $P = 0.983$ ， $S = 2.89 \times 0.983 \times 0.017 = 0.048$ 。前者信息量是后者的约 15 倍——所以选 $b = 0.5$ 。
- 作用：式 2 是 CAT 选题的总目标；式 3 说明打分函数形式复杂（分式、平方），不是点积，这正是不能直接建树的根源。

公式 5 & 6 + 算法 1: 学生感知超平面 (重点)

普通划分 (式 5、6): 超平面 $y(x) = w^\top x + b$, 其中 $w = B - A$, $b = -\frac{1}{2}(\|B\|^2 - \|A\|^2)$ 。这其实就是 A, B 两点的垂直平分面: $w^\top x + b \leq 0 \Leftrightarrow$ 点离 A 更近。

SECAT 的关键改动在怎么选 A, B (算法 1 的 MakeHyperplane):

$$\theta_l = \min_{\theta_i \in \Omega} \|\theta_i\|, \quad \theta_r = \max_{\theta_i \in \Omega} \|\theta_i\|$$

$$A = \arg \max_{q_i \in Q} S(\theta_l, q_i), \quad B = \arg \max_{q_i \in Q} S(\theta_r, q_i)$$

- 含义: θ_l, θ_r 是当前节点学生集合 Ω 里能力最低 / 最高的学生; A 是“最适合最弱学生的题”, B 是“最适合最强学生的题”。拿这两道题当质心切一刀。
- 举例: 节点里学生能力范围 $[-2, 2]$, $\theta_l = -2, \theta_r = 2$ 。在题库里: 最适合 $\theta = -2$ 的题是难度 $b = -2$ 的那道 $\rightarrow A$; 最适合 $\theta = 2$ 的是 $b = 2$ 那道 $\rightarrow B$ 。垂直平分面大致落在 $b = 0$ 处, 于是简单题 (适合弱生) 分左、难题 (适合强生) 分右。学生集合也对半分 (按 $\|\theta\|$ 中位数 θ_m), 递归继续。这就保证了“适合相似学生的题在同一叶子”。
- 作用: 让索引“学生感知”, 使得测试时一个学生只需进入和他能力匹配的少数叶子。消融实验 (图 6) 显示, 去掉它 (SECAT-s) 测试一开始就要搜更多叶子。

公式 7 & 8: 双塔蒸馏 (重点)

$$S(\theta, q) \leftarrow f(\theta)^\top g(q), \quad Loss = \sum_{j=0}^N \|f(\theta)^\top g(q_j) - S(\theta, q_j)\|^2$$

- 含义: 训练两个网络 f (用户塔)、 g (题目塔), 让它们输出向量的点积去逼近真实信息量 S 。损失就是 MSE。
- 举例: 取向量维度 15。题 $b = 0.5$ 对学生 $\theta = 0.4$ 的真值 $S = 0.717$ (上面算的)。训练目标就是让 $f([0.4, \text{历史}])^\top g([1, 0.5]) \rightarrow 0.717$ 。所有 (学生, 题) 对都这么拟合。训练完, “算信息量” $\sim$ “算点积”, 复杂度从复杂分式降到一次内积。
- 作用: 把无效的打分函数“翻译”成点积这个有效度量, 让 Ball Tree 的几何剪枝得以成立。

公式 9 / 11: 球内点积上界 (剪枝的命根子)

$$\max_{q_i \in B} f(\theta)^\top g(q_j) \leq f(\theta)^\top q_c + r \|f(\theta)\|$$

- 符号: 球 B 球心 q_c 、半径 r ; $f(\theta)$ 是查询向量。
- 推导直觉: 球内任一点 $g(q) = q_c + \Delta$, $\|\Delta\| \leq r$ 。则 $f(\theta)^\top g(q) = f(\theta)^\top q_c + f(\theta)^\top \Delta \leq f(\theta)^\top q_c + \|f(\theta)\| \cdot r$ (柯西-施瓦茨, Δ 与 $f(\theta)$ 同向时取等)。
- 举例: $f(\theta) = [3, 4]$ (模长 5), 球心 $q_c = [1, 1]$, 半径 $r = 0.5$ 。上界 $= 3 \times 1 + 4 \times 1 + 0.5 \times 5 = 7 + 2.5 = 9.5$ 。若我当前已找到的最佳信息量是 $10 > 9.5$, 那这个球里绝不可能有更好的题, 整球剪掉, 省去逐题计算。
- 作用: 分支定界的剪枝判据 (算法 2 第 6 行)。能不能大量剪枝, 决定了 L 多大、整体快不快。

公式 13 + 定义 1、2: 查询相似度与渐近查询 (多轮复用)

$$S \approx f(\hat{\theta})^\top g(q) = \|f(\hat{\theta})\| \|g(q)\| \cos \alpha$$

$$Sim(\theta_i, \theta_j) = \cos(f(\theta_i), f(\theta_j)); \quad \text{渐近查询: } \min_{\hat{\theta}_{t'} \in \Theta} Sim(\hat{\theta}_t, \hat{\theta}_{t'}) > \delta$$

- 关键观察: 点积排序只取决于夹角 α 和题向量长度, 查询向量的长度不影响选谁。所以两个学生估计“像不像”, 只看其向量夹角 $\rightarrow$ 用余弦衡量。
- 举例: $\delta = 0.95$ 。第 5 步 $\hat{\theta}_5$ 的向量与第 4 步 $\hat{\theta}_4$ 的向量夹角余弦 $= 0.97 > 0.95 \rightarrow$ 渐近查询成立 $\rightarrow$ 直接从第 4 步选过的那个叶子里挑题, 不搜整棵树。若余弦只有 $0.8 < 0.95 \rightarrow$ 说明能力估计还在大幅变化, 老老实实用算法 2 搜整棵树。
- 作用: θ 随测试推进而收敛 (式 12 的渐近一致性), 后期相邻步高度相似, 多轮复用让遍历叶子数 L 进一步降到近似常数。 δ 是效果/效率的旋钮: 太小 $\rightarrow$ 老复用旧叶子、可能错过好题 (ACC 降); 太大 $\rightarrow$ 老搜全树 (变慢)。图 7 显示需要折中, 论文取 0.95。

公式 12: 渐近一致性 (理论依据)

$$\lim_{n \rightarrow \infty} \Pr\{|\hat{\theta}_n - \theta_0| \geq \epsilon\} = 0$$

- 含义: 题答得越多, 能力估计 $\hat{\theta}_n$ 越收敛到真值 θ_0 (概率收敛)。
- 作用: 这是“多轮搜索能复用”的理论保证——既然 $\hat{\theta}$ 会越来越稳, 相邻步的查询就会越来越像, 复用旧叶子才合理。

时间复杂度小结 (4.4 节)

- 暴力: $O(ND)$ (D 是算一次 S 的成本)。
- 单轮 SECAT: $O(DL \log N)$ (树深 $\log N$, 遍历 L 个叶子)。
- 多轮 SECAT: L 降到近似常数 $\rightarrow$ 约 $O(D \log N)$ 。
- 直观数字: $N = 10$ 万时, 暴力算 10 万次 $\rightarrow$ SECAT 平均算 83 次。

五、代码实现对照

仓库 SECAT/CAT/, 几个关键对应:

- 双塔蒸馏 (公式 7、8) $\rightarrow$ CAT/distillation/MFI/model.py 的 distill 类:

```
self.utn = nn.Sequential(Linear(user_dim,256),Sigmoid,...,Linear(128,embedding_dim),Softplus) # 用户塔 f
self.itn = nn.Sequential(Linear(2,256),Sigmoid,...,Linear(128,embedding_dim),Softplus) # 题目塔 g (输入 2 维 = 难度 + 区分度)
def forward(self,u,i):
    return (self.utn(u) * self.itn(i)).sum(dim=-1) # 就是  $f(\theta)^T g(q)$ 
```

正好对应论文“双塔 + 点积近似信息量”。题目塔输入维度 = 2 (难度、区分度), 输出 embedding_dim (论文设 15)。

- 学生感知划分 (公式 5、6 + 算法 1) $\rightarrow$ CAT/mips/ball_tree.py 的 BallTree.make_metric_tree_split:

```
max_threshold = min(heapq.nlargest(threshold/2, thetas)) # 取能力高的一批
min_threshold = max(heapq.nsmallest(threshold/2, thetas)) # 取能力低的一批
if max_threshold > min_threshold and self.dissimilarity_partition:
    A = (适合高能力者的题 trait 的均值) # 对应论文里“最适合最强学生的题”
    B = (适合低能力者的题 trait 的均值)
else: # 退化为普通 Ball Tree: 随机点找最远点对
    A = data[argmax(dist to x)]; B = data[argmax(dist to A)]
w = B-A; b = -1/2*((B**2).sum()-(A**2).sum()) # 完全是式 (6)
```

dissimilarity_partition=True 时启用学生感知划分 (论文的 SECAT), False 时退化成传统 Ball Tree (对应消融的 SECAT-s)。注意代码用一批高/低能力学生取均值当 A, B , 比论文式子里“取单个 argmax 题”更稳健, 思想一致。

- 单轮分支定界搜索 (公式 9/11 + 算法 2) $\rightarrow$ ball_tree.py 的 search_metric_tree:

```
if res['quantity'] >= np.dot(q, T.center)+T.radius*((q**2).sum())**0.5:
    return 0 # 上界  $\leq$  当前最优  $\rightarrow$  剪枝, 正是公式 (9)
if T.is_leaf():
    for key,val in T.dict.items(): # 叶子内逐题精确算 (这里 dot 即蒸馏后点积)
        ... 更新 res['qid'], res['quantity'], res['leaves']
else:
    l_l=dot(T.left.center,q); l_r=dot(T.right.center,q)
    先递归上界大的子树 (对应算法 2 第 15-21 行的排序遍历)
```

返回值累加遍历的叶子数, 对应论文统计的 L 。

- 打分函数 / CDM: CAT/strategy/ 下 MFI_strategy.py (Fisher) 、KLI_strategy.py、MAAT_strategy.py、random_strategy.py; CAT/model/IRT.py、NCD.py。蒸馏前用这些原版 S 生成“top-k 标签”(CAT/distillation/tool.py 的 get_label_and_k, 按 MFI/KLI/MAAT 调用 get_k_fisher/get_k_kli/get_k_emc), 再训双塔去拟合。
- 多轮搜索 (算法 3): 体现在测试脚本/数据集对历史候选叶子的缓存复用 (CAT/dataset/adapttest_dataset.py + scripts/test*.py), 用相似度阈值 δ 决定走“复用旧叶子”还是“搜全树”。

六、小结

做了什么: 把 CAT 选题从 $O(N)$ 暴力穷举, 改造成 $O(\log N)$ 的树搜索, 三招——(1) 学生感知的 Ball Tree 划分 (让“适合同类学生的题”聚到一起); (2) 双塔蒸馏 (把复杂打分函数变成可剪枝的点积); (3) 多轮渐近复用 (后期直接复用旧叶子)。

效果: 两个真实大题库 (Eedi 2.7 万题、Exam 12 万题) 上, **200 倍以上加速** (如 Exam-NCD-MAAT 累积延迟从 14.95s $\rightarrow$ 0.77s), 而 ACC/AUC 与暴力几乎持平 (80.89 $\rightarrow$ 80.37 / 87.61 $\rightarrow$ 87.56), MSE 也基本不变。对比纯 ANN (ANNOY、HNSW), SECAT 精度明显更好, 尤其 MSE (0.178 vs 0.695/0.313) ——因为它在叶子内用原版精确打分函数, ANN 上界只在 Ball Tree 内有效。

优点：- 通用——可套在任何“最大信息量类”打分函数（FSI/KLI/MAAT）上，对 CDM（IRT/NCD）也无关。- 精度损失可忽略，真正能上大题库。

局限 / 留白：- 不适用于改变了 CAT 范式的数据驱动策略（NCAT、BOBCAT），因为它们不是“打分函数 + 搜索”的结构，且训练开销巨大（BOBCAT 在 Exam 上一个 epoch 18 小时）。- 需要离线训练双塔 + 建树，有额外的预处理成本（但只做一次，且测试时收益巨大）。- 没考虑曝光控制（exposure control）等现实约束——作者把这列为未来工作。 δ 需要按数据集调（论文取 0.95）。